

Explain 2 Keys Keyboard Code

The key idea is: every time you Copy All, you should think of it as creating a new block of characters that you can repeatedly paste. There are two good ways to solve this. For learning DP, I'll show you the memoization $\rightarrow$ tabulation approach, and then the much simpler mathematical solution.

1. Understand the problem first

Initially:

```
text
Screen = "A"
```

So we already have 1 A.

For $n = 3$:

```
text
A
|
├─ Copy All
└─ Paste → AA
    |
    └─ Paste → AAA
```

Operations:

```
text
Copy All
Paste
Paste
```

Answer = 3.

For $n = 1$, we don't need anything.

```
text
Answer = 0
```

2. DP intuition

Suppose we currently have i characters.

If we perform:

```
text
Copy All
```

the clipboard contains i characters.

Now every paste adds another i .

So after one copy, we can reach:

```
text
i → 2i → 3i → 4i → ...
```

The important observation is:

We can reach n from i in one Copy + some number of Pastes only if n is divisible by i .

For example, $n = 12$.

From 3:

```
text
AAA
Copy
Paste → 6
Paste → 9
Paste → 12
```

Cost:

```
text
1 Copy + 3 Paste = 4 operations
```

So:

text

```
dp[12] = dp[3] + 4
```

And perhaps:

text

```
dp[3] = 3
```

Therefore:

text

```
dp[12] = 3 + 4 = 7
```

3. Memoization

Define:

text

```
solve(n)
```

as:

Minimum operations required to produce exactly n A's starting from one A.

Instead of directly thinking about all Copy/Paste sequences, we look for the **last Copy**.

Suppose the last Copy happened when we had d characters.

After that Copy, we paste enough times to reach n .

If:

text

```
n = d * k
```

then:

- Copy = 1 operation
- Pastes = $k - 1$ operations

Total:

text

```
1 + (k - 1) = k
```

Therefore:

text

```
solve(n) = solve(d) + n/d
```

for every divisor d of n .

Memoization code

C++

```
class Solution {
public:
    int solve(int n, vector<int>& dp) {
        if (n == 1) return 0;

        if (dp[n] != -1) return dp[n];

        int ans = n; // Worst case: Copy + Paste n-1 times

        for (int d = 1; d < n; d++) {
            if (n % d == 0) {
                ans = min(ans, solve(d, dp) + n / d);
            }
        }

        return dp[n] = ans;
    }

    int minSteps(int n) {
        vector<int> dp(n + 1, -1);
        return solve(n, dp);
    }
};
```

4. Dry run for n = 6

Let's see exactly how this works.

We want:

```
text
solve(6)
```

Possible divisors:

```
text
1, 2, 3
```

Choose d = 1

From 1:

```
text
Copy → Paste → Paste → Paste → Paste → Paste
```

That's:

```
text
solve(1) + 6/1
= 0 + 6
= 6
```

Choose d = 2

First solve 2.

```
text
solve(2)
```

From 1:

```
text
Copy
Paste
```

So:

```
text
solve(2) = 2
```

Now from 2 to 6:

```
text
AA
Copy
Paste → AAAA
Paste → AAAAAA
```

Cost:

```
text
2 pastes + 1 copy = 3
```

Therefore:

```
text
solve(6)
= solve(2) + 6/2
= 2 + 3
= 5
```

Choose d = 3

```
text
solve(3) = 3
```

From 3 to 6:

```
text
AAA
Copy
Paste → AAAAAA
```

Cost:

```
text
2
```

Therefore:

```
text
solve(6) = solve(3) + 6/3
         = 3 + 2
         = 5
```

Final:

```
text
dp[6] = 5
```

One optimal sequence:

```
text
A
Copy
Paste
Copy
Paste
Paste
```

Wait — let's count carefully:

```
text
A
Copy
Paste → AA
Copy
Paste → AAAA
Paste → AAAAAA
```

That's:

```
text
Copy
Paste
Copy
Paste
Paste
```

5 operations.

5. Tabulation

Memoization is top-down:

```
text
solve(n)
  ↓
solve(smaller problem)
  ↓
solve(smaller problem)
```

Tabulation does the opposite.

We calculate:

```
text
dp[1]
dp[2]
dp[3]
...
dp[n]
```

Code

C++

```
class Solution {
public:
    int minSteps(int n) {
        if (n == 1) return 0;

        vector<int> dp(n + 1, 0);

        for (int i = 2; i <= n; i++) {
            dp[i] = i;

            for (int d = 1; d < i; d++) {
                if (i % d == 0) {
                    dp[i] = min(dp[i], dp[d] + i / d);
                }
            }
        }

        return dp[n];
    }
};
```

6. How the tabulation works

For $n = 6$:

i	dp[i]	Explanation
1	0	Already have A
2	2	1 → 2
3	3	1 → 3
4	4	Or 1 → 2 → 4, which costs 4
5	5	Prime number
6	5	1 → 2 → 6 or 1 → 3 → 6

For 6:

text

dp[6]

checks:

text

$d = 1 \rightarrow dp[1] + 6/1 = 0 + 6 = 6$

$d = 2 \rightarrow dp[2] + 6/2 = 2 + 3 = 5$

$d = 3 \rightarrow dp[3] + 6/3 = 3 + 2 = 5$

So:

text

dp[6] = 5

7. But there is a MUCH simpler solution

This problem has a mathematical observation.

Every time you perform a useful Copy + Paste sequence, you're effectively **multiplying** the number of A's.

For example:

text

1 → 2

requires:

text

2 operations

because:

text

Copy + Paste

And:

text

2 → 6

requires:

text

Copy + Paste + Paste

which costs 3.

So:

text

1 → 2 → 6

cost:

text

2 + 3 = 5

Notice the factors:

text

6 = 2 × 3

Therefore, the answer is simply:

Sum of the prime factors of n .

For example:

text

$n = 12$

$12 = 2 \times 2 \times 3$

answer = 2 + 2 + 3
= 7

Sequence:

text

1
↓ Copy + Paste
2
↓ Copy + Paste
4
↓ Copy + Paste + Paste
12

Total:

text

2 + 2 + 3 = 7

8. Optimal code

C++

```
class Solution {
public:
    int minSteps(int n) {
        int ans = 0;

        for (int d = 2; d * d <= n; d++) {
            while (n % d == 0) {
                ans += d;
                n /= d;
            }
        }

        if (n > 1) {
```

```

        ans += n;
    }

    return ans;
}
};

```

Example: `n = 18`

Factorize:

```

text
18
↓ /2
9
↓ /3
3
↓ /3
1

```

So:

```

text
18 = 2 × 3 × 3

```

Answer:

```

text
2 + 3 + 3 = 8

```

Operations:

```

text
A
Copy + Paste      → AA

Copy + Paste      → AAAA

Copy + Paste
Paste              → AAAAAAAAAA

```

Actually, that last stage goes `4 → 12`, not `18`; the correct factor sequence for `18` is:

```

text
1 → 2 → 6 → 18

```

with costs:

```

text
2 + 3 + 3 = 8

```

The core pattern to remember

This is the important part for DSA:

```

text
Number of A's
↓
Choose a previous divisor
↓
Copy once
↓
Paste n/d - 1 times
↓
Cost = n/d

```

So the DP recurrence is:

```

text
dp[n] = min(dp[d] + n/d)

```

where:

```

text
d divides n

```

And because the problem has a special mathematical structure, it simplifies to:

text

```
answer = sum of prime factors of n
```

Complexity

DP solution:

text

Time: $O(n^2)$

Space: $O(n)$

Prime-factor solution:

text

Time: $O(\sqrt{n} \log n)$ *// due to repeated division*

Space: $O(1)$

For LeetCode, I'd use the **prime-factor solution**, but understanding the DP recurrence is valuable because it teaches you how to recognize "**build the target through divisors/factors**" problems.